



## PROBLEM STATEMENT

## Personalized Adaptive Learning &amp; Assessment Orchestrator

# CogniFlow

Most tutors make the next question easier.  
CogniFlow finds out why you failed, and teaches that first.

An adaptive learning agent that traces a wrong answer back to the **missing prerequisite**, repairs it, and brings the student back to their **original goal**.

```
student_answer.py

def total(n):
    if n == 0:
        return 0
    n + total(n - 1) # never returned

TypeError: unsupported operand type(s) for +: 'int'
and 'NoneType'
```

Looks like a recursion problem. It's a **functions** problem: the student doesn't yet know what return does.

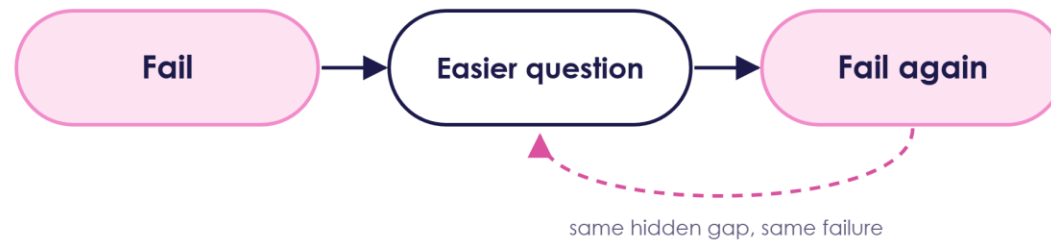
## THE PROBLEM

# Easier questions can't fix a missing foundation

When a student keeps failing, adaptive systems lower the difficulty. But the real gap is often **one level down**: a student failing recursion may not know what `return` does. The easier question hits the same wall.

## TRADITIONAL

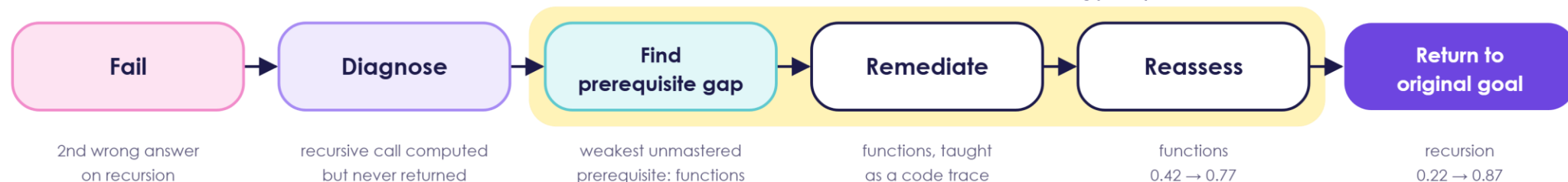
turns the difficulty dial



Difficulty goes down.  
The missing skill never comes up.

## COGNIFLOW

turns the goal dial



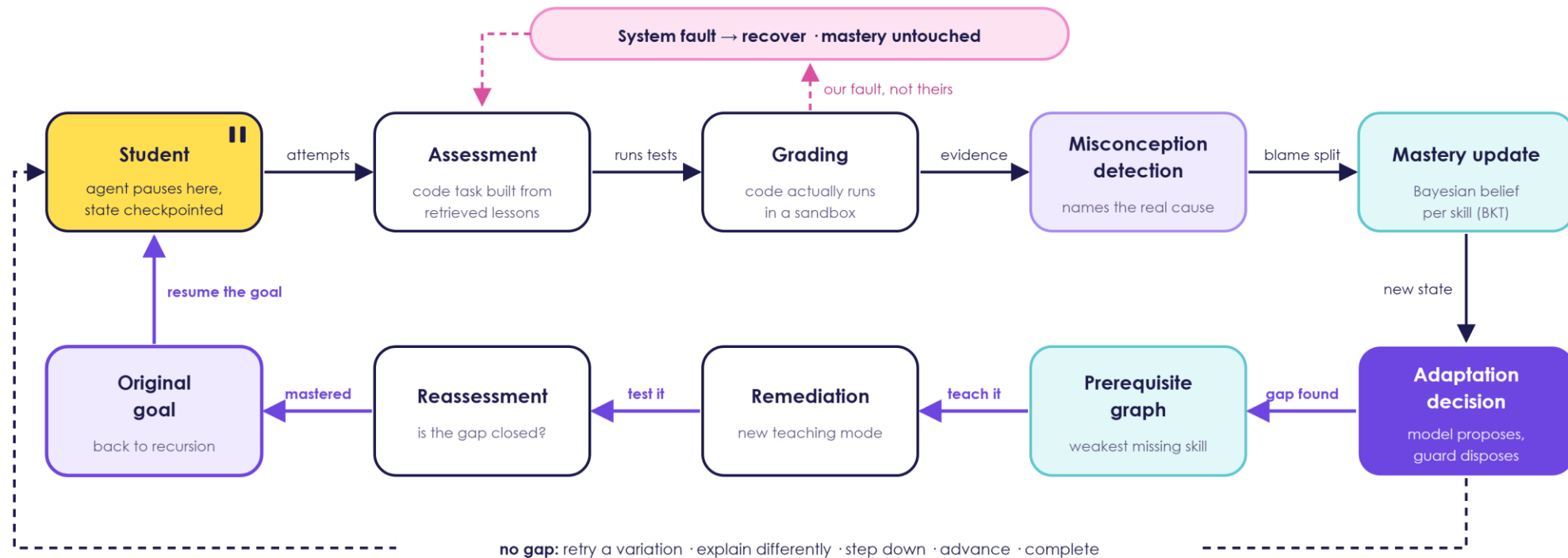
Difficulty is the wrong dial. The right one is what the skill is built on.



## THE ADAPTIVE LOOP

# It re-reads the student after every answer

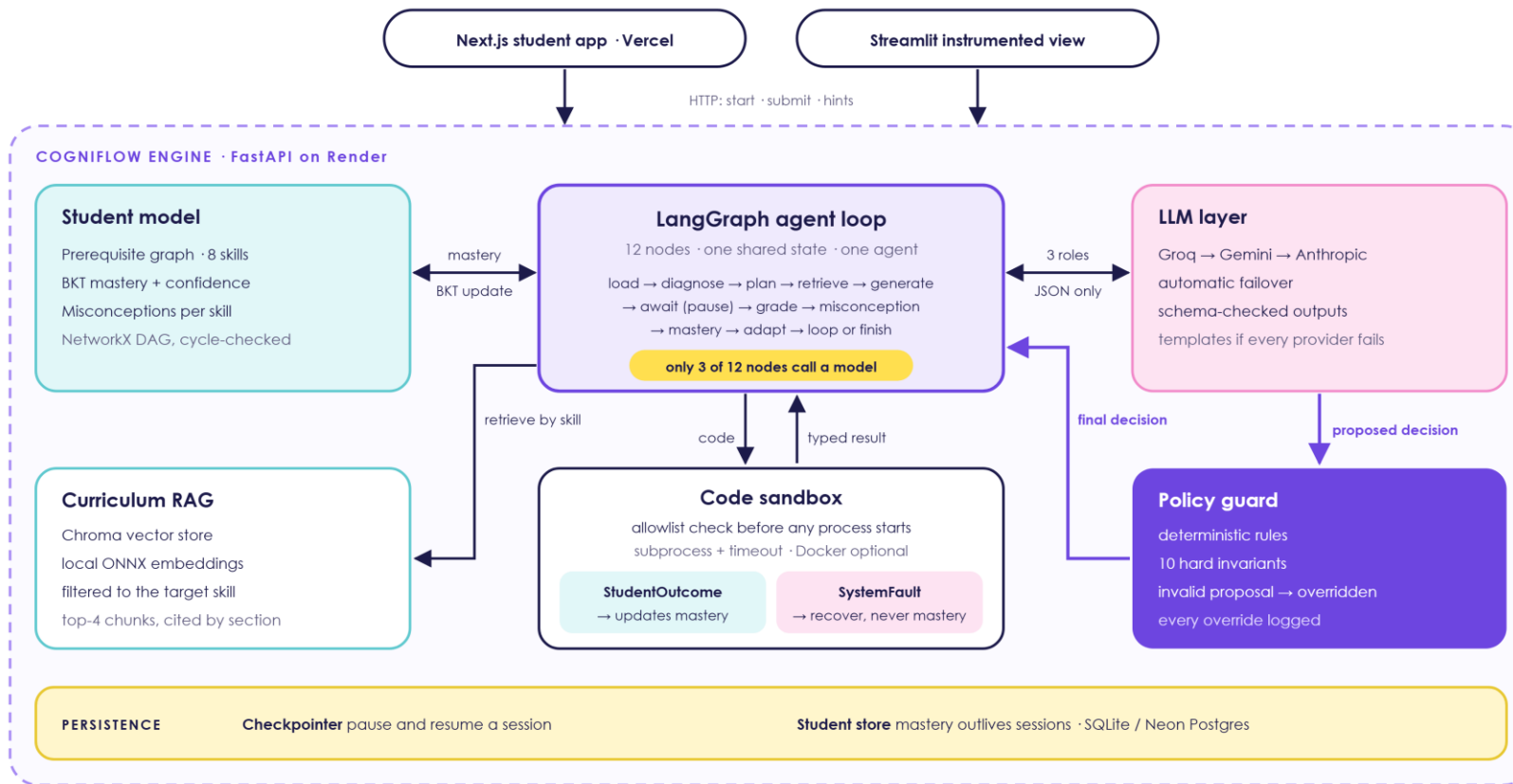
No path is scripted. Each pass updates what the agent believes about the student (**mastery**, **confidence**, **misconceptions**) and picks the next move from eight possible actions.



Observe → update belief → decide → act → observe again, until the original goal is mastered.

## ARCHITECTURE

# One agent. The model proposes, the guard disposes.



## WHY IT'S GENUINELY AGENTIC

**It changes its own goal.** The target skill moves from recursion to functions and back. Nobody scripted that path.

**It loops over state it keeps.** Every answer updates a persistent belief, and that belief drives the next decision.

**It observes through tools.** It runs code, retrieves curriculum and walks a skill graph, then acts on what comes back.

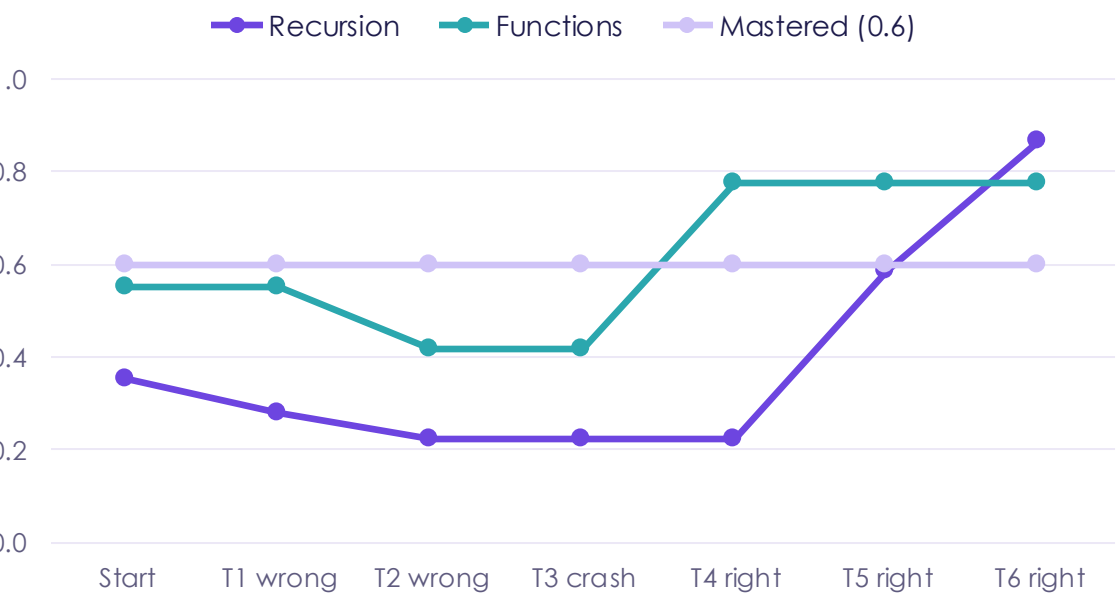
**It pauses and resumes.** It checkpoints mid-session while the student thinks.

Not multi-agent, by design: one goal, one shared state. Routing, mastery and prerequisite choice are deterministic and unit-tested.

RESULTS

# Measured, not claimed

One real session. A redirect, a crash, a return.



Recursion 0.35 → 0.87 · functions 0.55 → 0.77 · turn-3 crash moved nothing

7 / 7 self-checks pass

558 tests pass offline

Retrieval recall@4: 100%

BKT fitting: negative result, reported

## Does the prerequisite graph help?

80 simulated students per arm · half with a planted gap

CogniFlow policy      Same tutor, prerequisite graph removed

### Planted gap found

of students who had the hidden gap



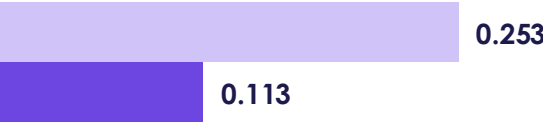
### Attempts to mastery

median tries · lower is better



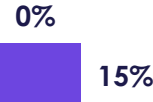
### Estimate error

belief vs true skill · lower is better



### Unneeded detours

no-gap students sent back · the honest cost

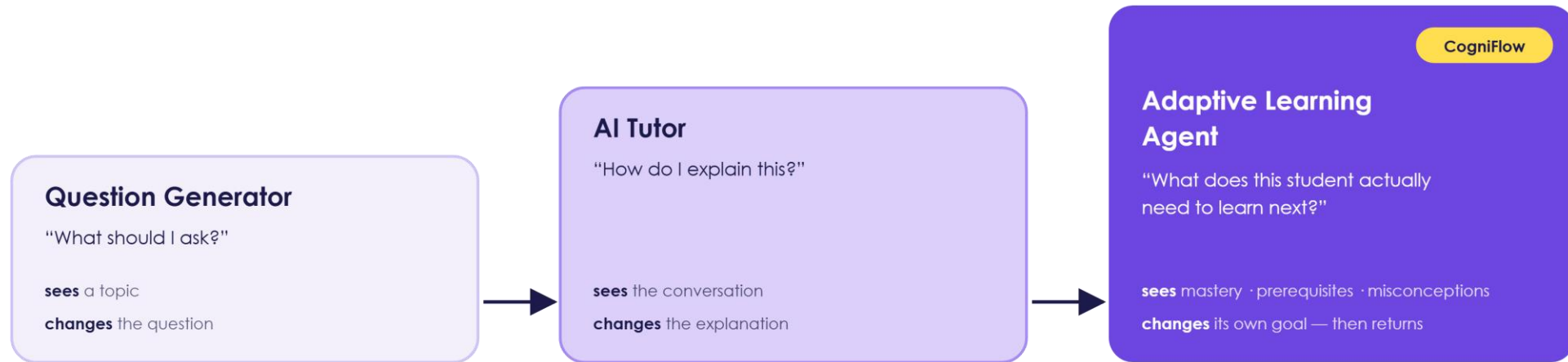


Reproduce: python run.py ablation



## WHY IT MATTERS

# From asking questions to finding what's missing



**Explainable.** Every move is logged with its reason and evidence, so a teacher can see which prerequisite blocked which student.

**Safe.** Our outage can never lower a student's score. The types make it impossible.

**Portable.** The skill graph and curriculum are data files. A new subject starts with new data.

## A wrong answer isn't a score. It's a clue.

CogniFlow follows it to what the student actually needs to learn next.

